

Exam Guide

Automatic Image Analysis ■ learning from exam mistakes

Every mistake turned into a rule, a procedure, and a picture

How to use this guide

The mistakes are grouped by topic, not by exam. Each chapter collects the errors that belong to one subject area. Every error follows the same pattern: the **task**, the **mistake**, the **correct answer**, **why** it is correct, a **step-by-step** procedure that works on any variant of the task, and a short **rule to remember**. Chapter 1 comes first because its habits prevent errors in every other chapter; Chapter 11 condenses everything into one checklist.

Contents

- 1 Answering multiple-choice questions
- 2 Vision tasks and the processing chain
- 3 Histograms, entropy, and saliency
- 4 Hough transform and template matching
- 5 Bayesian decision theory
- 6 Parameter estimation
- 7 Training neural networks
- 8 Convolutional networks
- 9 Recurrent networks
- 10 Generative adversarial networks
- 11 Checklist before submitting

1 Answering Multiple-Choice Questions

Most lost points came from multi-select questions: one option too many or one too few. The knowledge was usually there; the checking procedure was not. The same few traps appear again and again, so a fixed routine per option is worth more than any single fact.

Four recurring traps

Only one term

A risk computed from one product instead of the sum; a BPTT derivative along one path instead of all.

Absolute words

“any function”, “always the lowest MSE”, “fewer steps”: one counterexample makes them false.

Half the question

“true for CNNs *but not* MLPs” needs two checks. “Underfitting” is not “overfitting”.

Missed a true option

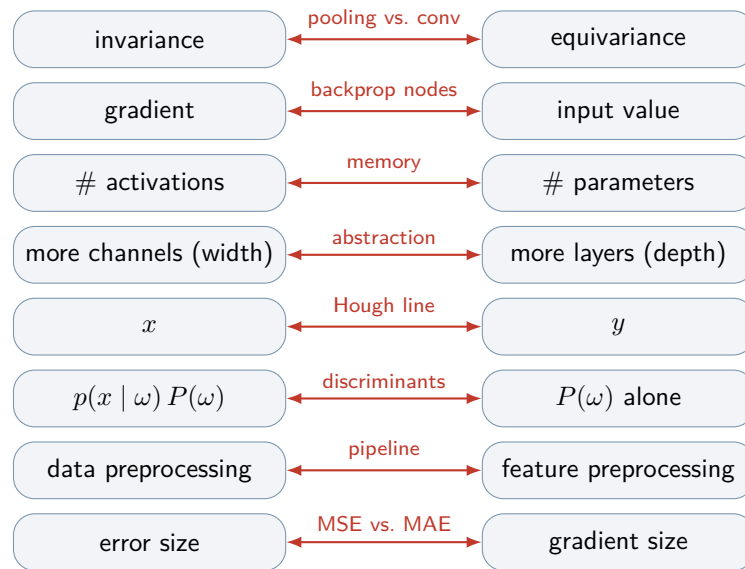
The last pipeline step, a second task of a system, a textbook fact that looked too easy.

The four traps behind most lost points. The first costs points in numeric tasks, the other three in multi-select tasks.

Step by step

1. **Judge every option on its own.** Cover the others and ask: “Is this sentence true by itself?” Never stop after the first true option, and never assume that exactly one or two options are correct.
2. **Look for absolute words.** For “always”, “only”, “any”, “necessarily”, “fewer”, “lower”, search for one counterexample. One is enough to make the statement false.
3. **Two-part questions.** “True for A but not for B” needs two checks per option: true for A? *and* false for B?
4. **Check for swapped partner concepts** (figure below). The trap is a true sentence with one word exchanged for its partner.
5. **Check that the option answers the question.** “Higher abstraction” is a true property of CNNs, but not an effect of *more filters in one layer*.
6. **Numeric answers: formula first, numbers second.** Write the formula with its $\sum$ and count the terms before inserting numbers.
7. **Sanity check the result.** A probability lies in $[0, 1]$; a risk lies between the smallest and largest loss of that action; check the order of magnitude.

Partner concepts



Pairs of concepts that were confused. When an option contains one of them, re-read it with the partner and decide which one makes the sentence true. The small label names the chapter where the pair matters.

2 Vision Tasks and the Processing Chain

A task is defined by what it outputs, and a pipeline stage by what goes in and what comes out. Every error in this chapter disappears once the output type is written down before choosing an answer.

Which tasks does a system solve?

Question

Basketball software: bounding boxes around players, identifies name and number, describes the action (dribbling, throwing, defense).

✗ Mistake

Only Action Recognition.

✓ Correct answer

Object Recognition *and* Action Recognition.

Step by step

1. List every output the system produces: box, name/number, action.
2. Map each output to a task with the table below.
3. Tick **every** task that owns one of the outputs. A system usually combines several tasks.

Output	Task	Here?
one label for the whole image	object classification	✗ no
box + <i>which specific</i> player (name, number)	object recognition (identity)	✓
one class per pixel	semantic segmentation	✗ no pixel map
verb over time (dribbling)	action recognition	✓

Remember

Classification = which *category* (“a player”). **Recognition** = which *individual* (“player 23, Smith”).

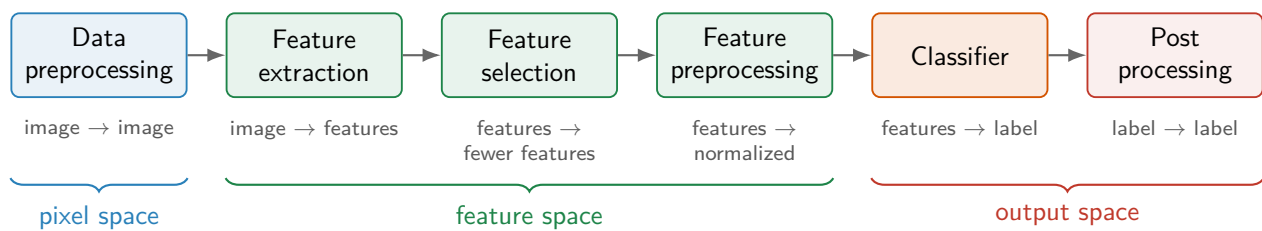
Order of the processing chain

Question

Bring Data Preprocessing, Feature Selection, Feature Extraction, Feature Preprocessing, Post processing, and Classifier into the correct order.

✓ Correct answer

Data Preprocessing → Feature Extraction → Feature Selection → Feature Preprocessing → Classifier → Post processing.



The six stages, sorted by what goes in and what comes out. Everything in pixel space comes first; everything that needs labels comes last.

Step by step

1. Write down input and output type of each stage (image, features, label).
2. Chain them so that each output type is the next input type.
3. Inside feature space: you can only *select* features that have been *extracted*; you normalize only the features that survive the selection, directly before the classifier.

Remember

Data → Extract → Select → Prepare → Classify → Post.

Histogram matching is data preprocessing

Question

Images taken under different illumination are transformed so that all histograms have approximately the same distribution. Which pipeline stage is this?

✗ Mistake

Feature preprocessing.

✓ Correct answer

Data preprocessing.

Why

The method changes pixel values so that all images get a similar histogram. Input: image. Output: image. That is the signature of data preprocessing (pixel space). Feature preprocessing also “normalizes”, but it acts on *feature vectors* after feature extraction, for example scaling each feature to zero mean and unit variance. The word “normalize” is the trap; the input/output type decides.

Remember

Ask: *what comes out?* Image ⇒ data preprocessing. Feature vector ⇒ extraction or feature preprocessing. Label ⇒ classifier or postprocessing.

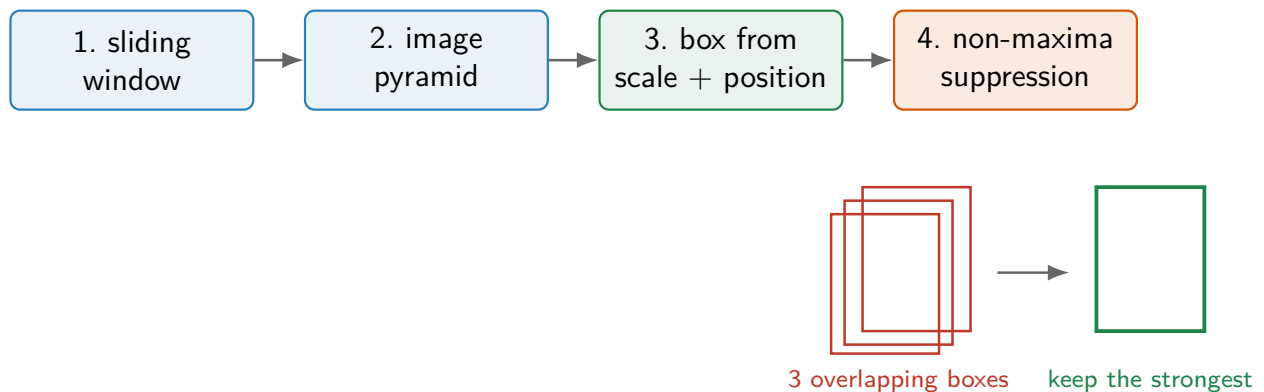
From classifier to detector: four steps

Question

A classifier works for one fixed image resolution. Which steps turn it into an object detector?

✗ Mistake

Sliding window, image pyramid, and boxes from scale and response were ticked; non-maxima suppression was missed.



A fixed-size classifier becomes a detector in four steps. Neighboring windows and pyramid levels all fire on the same object, so without step 4 one produces a cluster of boxes.

Remember

Detector = **slide**, **scale**, **box**, **suppress**. All four belong to the answer.

3 Histograms, Entropy, and Saliency

Entropy is computed from the histogram alone, and the histogram forgets where the pixels are. Both mistakes in this chapter come from forgetting one of these two facts: either the spatial arrangement was taken into account where it does not matter, or saliency was reduced to entropy where scale does matter.

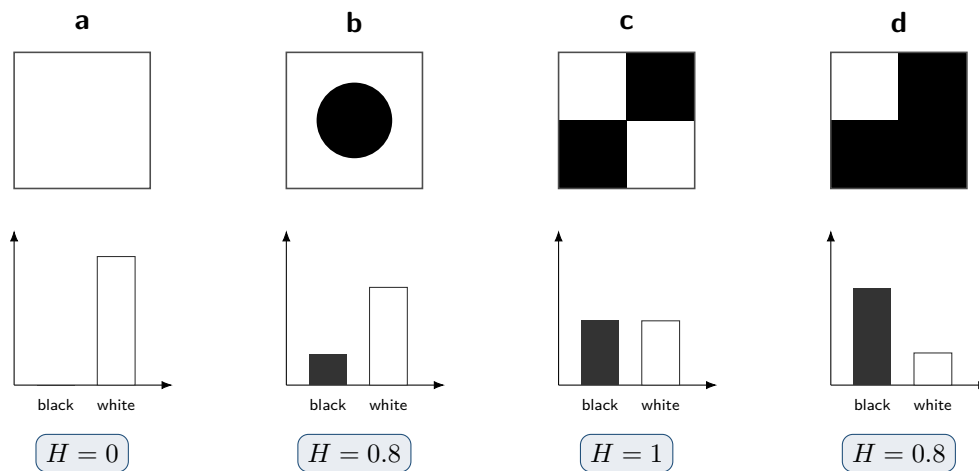
Matching histograms and entropy values

Question

Four black-and-white patches: assign each its histogram and its entropy value.

✗ Mistake

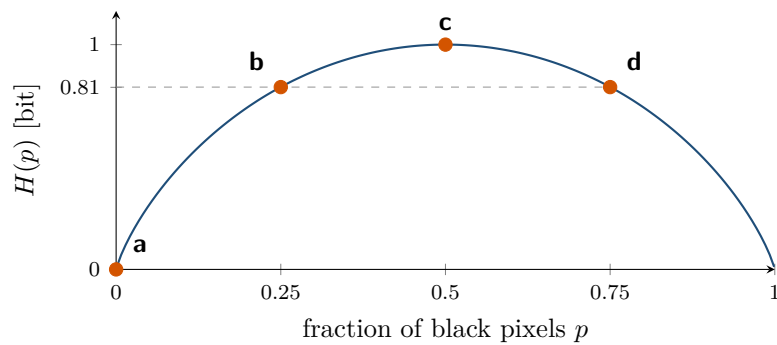
One pair was swapped: two patches with mirrored histograms (mostly white vs. mostly black) were exchanged.



Correct matching. The histogram counts only *how many* black and white pixels there are, never *where* they are. Patches b and d have mirrored histograms but the same entropy.

Step by step

1. **Fix the axis orientation first.** The single-bar histogram must belong to the all-white patch a. Its bar sits on the right, so the axis runs from black (left) to white (right).
2. For each patch, estimate the black/white fraction and pick the histogram with those bar heights. Mostly white (b): tall bar right. Mostly black (d): tall bar left.
3. Compute $H = -\sum_i p_i \log_2 p_i$: one value $\Rightarrow 0$; 50/50 $\Rightarrow 1$; 25/75 $\Rightarrow 0.81$.
4. The two “0.8” tokens look identical but may be graded as distinct items. Place each next to the histogram it belongs to.



Binary entropy is symmetric: 25 % black and 75 % black give the same 0.81 bit.

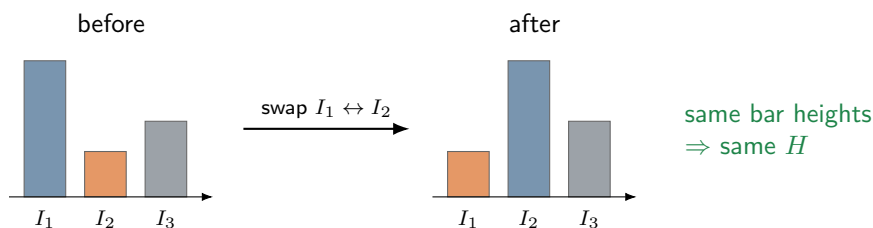
Swapping two grey values keeps the entropy

Question

Which statements about entropy are true? (Uniform histogram, swapping two colors, permuting pixels, homogeneous patch vs. edge.)

✗ Mistake

Only “uniform histogram $\Rightarrow$ highest entropy” was ticked; “swapping two colors does not change the entropy” was missed.



Entropy $H = -\sum_k p_k \log_2 p_k$ is a sum over the bar heights. Swapping two grey values moves two bars but keeps the same set of heights, so every term of the sum survives.

Statement	True?	Reason
uniform histogram $\Rightarrow$ highest entropy	✓	flattest possible distribution
swap two colors $\Rightarrow$ entropy unchanged	✓	same heights, other positions
permute pixels $\Rightarrow$ entropy changes	✗	histogram unchanged
homogeneous patch $>$ edge patch	✗	homogeneous $H = 0$

Remember

Entropy only sees the **list of bar heights**. Any change that keeps this list (relabel colors, shuffle pixels, rotate, mirror) keeps H .

Most salient patch after Kadir et al.

Question

Which of the four patches above is the most salient after Kadir and Brady?

✗ Mistake

Patch c, the checkerboard, because it has the highest entropy.

✓ Correct answer

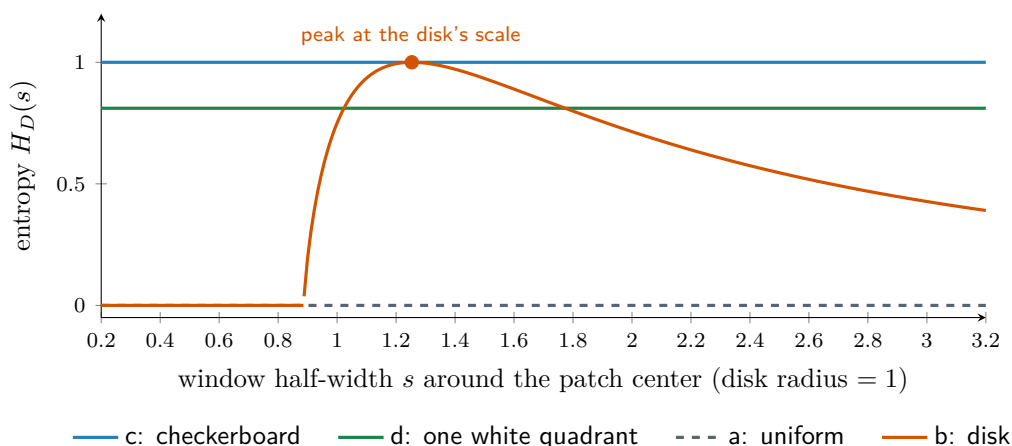
Patch b, the black disk.

Why

Kadir–Brady saliency is **not** entropy alone:

$$y(s_p, x) = H_D(s_p, x) \cdot W_D(s_p, x), \quad W_D(s, x) = s \int \left| \frac{\partial}{\partial s} p(I, s, x) \right| dI.$$

H_D is taken at a scale s_p where the entropy has a **peak** over scale, and W_D is large only if the histogram **changes across scale**. A patch whose statistics look the same at every window size scores zero, however high its entropy.



Entropy as a function of window size, window centered on each patch. Checkerboard, quadrant, and uniform patch keep the same black/white ratio at every size: flat curves, no peak, $W_D = 0$. Only the disk shows a peak where the window just encloses it; there the histogram changes fastest.

Step by step

1. Grow a window around the center of each patch and imagine its histogram.
2. Does the histogram change with window size? If not, the entropy curve is flat: no peak, not salient (a, c, d).
3. Is there a window size where the entropy is maximal and then falls again? That is a *characteristic scale*: salient (b).

Remember

Kadir saliency favors **isolated blobs with a characteristic scale**, not the maximum-entropy patch. Entropy alone is insufficient because the intensity distribution contains **no spatial information**.

4 Hough Transform and Template Matching

All line-detection tasks rest on one equation, $\rho = x \cos \theta + y \sin \theta$. The mistakes came from matching numbers by eye instead of inserting them, and from forgetting how much a known gradient constrains a vote. Template matching by correlation closes the chapter.

Does a line pass through a point?

Question

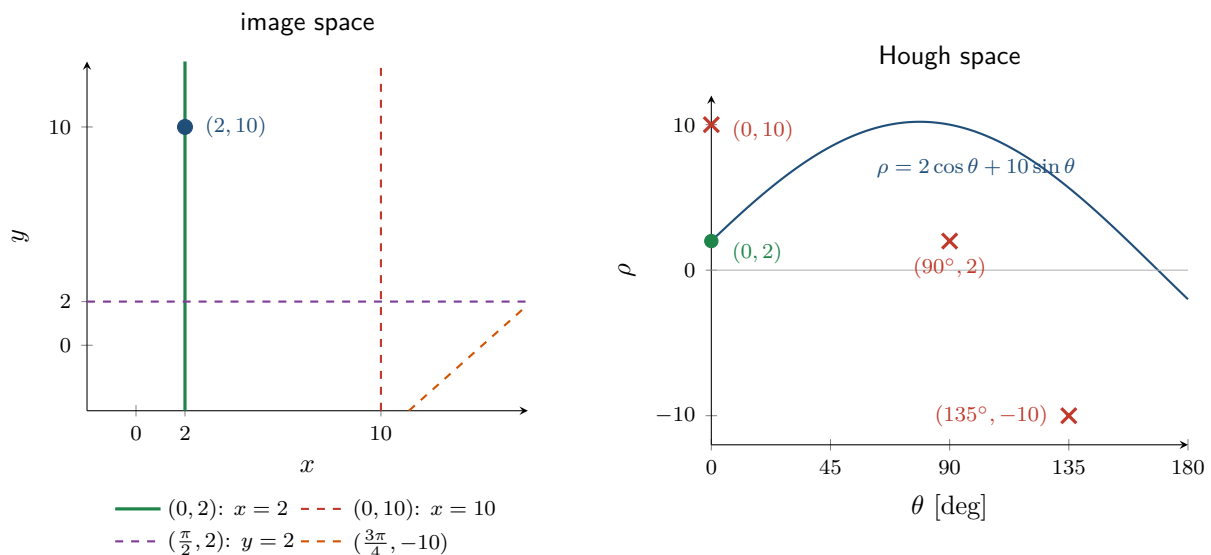
Which lines (θ, ρ) pass through $(x, y) = (2, 10)$? Candidates: $(0, 2)$, $(0.75\pi, -10)$, $(0, 10)$, $(0.5\pi, 2)$.

✗ Mistake

$(0, 10)$ and $(0.5\pi, 2)$: the roles of x and y were swapped.

✓ Correct answer

Only $(\theta, \rho) = (0, 2)$.



Left: the four candidate lines; only $x = 2$ passes through the point. Right: in Hough space the point becomes a sinusoid; a line passes through the point exactly when its (θ, ρ) lies on the curve.

Step by step

1. Write the line equation: $\rho = x \cos \theta + y \sin \theta$.
2. Insert the point: $\rho(\theta) = 2 \cos \theta + 10 \sin \theta$.
3. Evaluate every candidate and compare with the given ρ :

Candidate	$\cos \theta$	$\sin \theta$	$2 \cos \theta + 10 \sin \theta$	given ρ	
$(0, 2)$	1	0	2	2	✓
$(0.75\pi, -10)$	-0.707	0.707	5.66	-10	✗
$(0, 10)$	1	0	2	10	✗
$(0.5\pi, 2)$	0	1	10	2	✗

Remember

$\theta = 0$: $\rho = x$, the line is **vertical** ($x = \rho$). $\theta = \pi/2$: $\rho = y$, the line is **horizontal** ($y = \rho$). Always compute; never match numbers by eye.

How many votes does a point with a gradient cast?**Question**

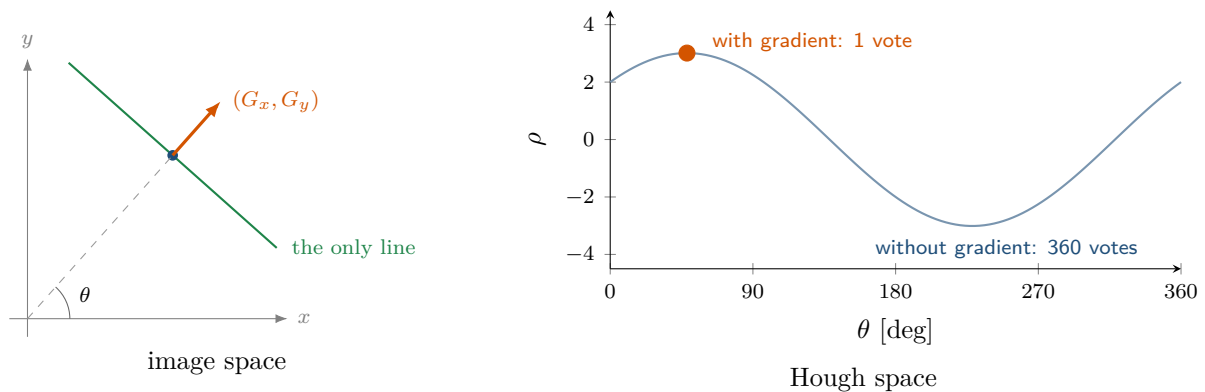
$\theta \in \{0^\circ, \dots, 359^\circ\}$, $\rho \in \{-100, \dots, 100\}$. Given a point (x, y) and its gradient (G_x, G_y) : how many (θ, ρ) candidates, i.e. how many votes?

✗ Mistake

360 (one vote per θ).

✓ Correct answer

1.



Without a gradient the point is consistent with every line through it: one vote per θ bin, a whole sinusoid. The gradient is the line's normal, so it fixes θ ; then $\rho = x \cos \theta + y \sin \theta$ is fixed too, and exactly one cell receives the vote.

Step by step

1. Point only: θ free, ρ follows $\Rightarrow$ one vote per θ bin (here 360).
2. Point *and* gradient: $\theta = \text{atan2}(G_y, G_x)$ fixed $\Rightarrow \rho$ fixed $\Rightarrow$ **1** vote.
3. The number of ρ bins (201) never counts: ρ is always computed, never searched.

Remember

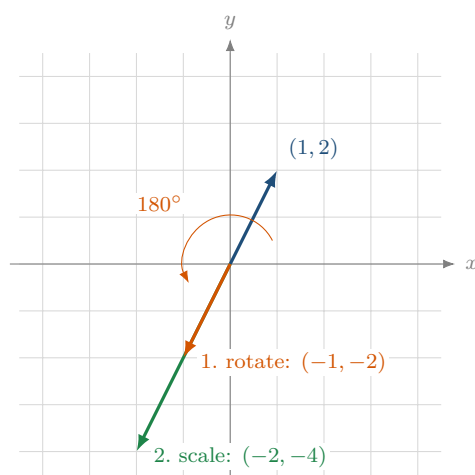
Count the **free parameters** after using all given information. Each free parameter multiplies the votes by its number of bins; zero free parameters means one vote.

Transforming an R-table

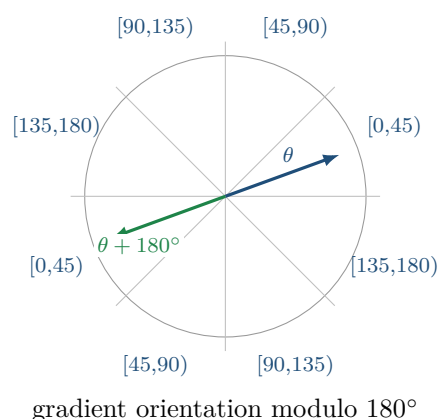
This task was solved correctly, but it has several traps. The procedure below keeps it reproducible for other scales, angles, and table ranges.

Question

Transform the R-table with scale $s = 2$ and a counter-clockwise rotation of $\varphi = 180^\circ$. Rows: $[0, 45)$: (1, 2); (3, 4) $[45, 90)$: – $[90, 135)$: (5, 6); (7, 8) $[135, 180)$: (9, 10).



displacement vectors



Left: rotating by 180° negates each displacement, scaling by 2 doubles it. Right: a gradient direction rotated by 180° lies on the same line, so with a table over $[0^\circ, 180^\circ)$ it stays in its row.

Step by step

1. **Scale:** multiply every displacement by s . Rows stay, since scaling does not change gradient directions.
2. **Rotate the vectors** by φ . For 180° : $(d_x, d_y) \mapsto (-d_x, -d_y)$.
3. **Rotate the rows:** every entry moves to the row for $\theta + \varphi$. Check the table range! Over $[0^\circ, 180^\circ)$, orientations are taken modulo 180° , so $\theta + 180^\circ \equiv \theta$: nothing moves.
4. Result: $(1, 2) \rightarrow (-2, -4)$, $(3, 4) \rightarrow (-6, -8)$, $(5, 6) \rightarrow (-10, -12)$, $(7, 8) \rightarrow (-14, -16)$, $(9, 10) \rightarrow (-18, -20)$; rows unchanged.

Remember

Typical distractors: only shifted rows, swapped components (= reflection), or an unchanged table. If the table covers $[0^\circ, 360^\circ)$, a 180° rotation *does* move rows. Translation never changes an R-table.

Gradient correlation and the correlation theorem

Question

Instead of R-tables, correlate the complex gradient images $I' = \partial_x I + i \partial_y I$ and $O' = \partial_x O + i \partial_y O$. Which statements are true about $\text{Re} / \text{Im}(I' \star O')$ and about computing $I' \star O'$ with the DFT?

✗ Mistake

The real part was chosen correctly, but also $I' \star O' = \text{DFT}^{-1}(\text{DFT}(I) \cdot \text{DFT}(O))$, which has no conjugate and therefore computes a convolution.

✓ Correct answer

$\text{Re}(I' \star O')$ sums the gradient dot products (almost Hough voting), and $I' \star O' = \text{DFT}^{-1}(\text{DFT}(I) \cdot \text{DFT}(O)^*)$.

Frequency-domain product	Signal-domain result
$\text{DFT}(f) \cdot \text{DFT}(g)$	convolution $f * g$
$\text{DFT}(f) \cdot \text{DFT}(g)^*$ or $\text{DFT}(f)^* \cdot \text{DFT}(g)$	correlation (the two only mirror the shift)
$\text{DFT}(f)^* \cdot \text{DFT}(g)^*$	neither

Why

Correlation slides the template *without flipping* it; convolution flips it. In the frequency domain, flipping a real signal is complex conjugation. The conjugate undoes the flip that convolution would apply, so exactly one conjugate turns a convolution into a correlation. For the real part: with $a = a_x + ia_y$, $b = b_x + ib_y$, $\text{Re}(a^*b) = a_x b_x + a_y b_y$, the dot product of the two gradient vectors. The imaginary part is a cross product.

Remember

Conv = plain product. **Corr** = product with **one star**.

5 Bayesian Decision Theory

Every task in this chapter is a sum or a ranking. Posteriors and risks are sums over *all* hypotheses or states; discriminant functions only need to rank the classes like the posterior; the expected error integrates x away. The mistakes came from dropping terms from a sum and from overlooking standard facts about Gaussian classes.

Computing a posterior

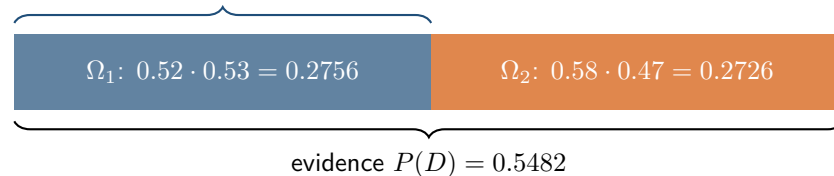
Question

$P(D | \Omega_1) = 0.52$, $P(D | \Omega_2) = 0.58$, $P(\Omega_1) = 0.53$, $P(\Omega_2) = 0.47$. Find $P(\Omega_1 | D)$ to four decimals.

✓ Correct answer

0.5027

$$P(\Omega_1 | D) = 0.2756 / 0.5482 = 0.5027$$



Each bar is likelihood $\times$ prior. The posterior is one bar's share of the total. Ω_2 has the higher likelihood, but its lower prior makes Ω_1 slightly more probable.

Step by step

1. Numerator per hypothesis: likelihood $\times$ prior.
2. Evidence: sum of *all* numerators.
3. Posterior: own numerator / evidence. Check that all posteriors sum to 1 ($0.5027 + 0.4973$).
4. Keep four or more decimals in between; round only at the end.

Expected risk of an action

Question

Loss $\lambda(\alpha_i | \omega_j)$: treat (α_1) costs 18 if ill (ω_1) and 27 if healthy (ω_2). $P(\omega_1 | x) = 0.75$. Expected risk of treating?

✗ Mistake

$6.75 = 27 \cdot 0.25$: only the “healthy” term.

✓ Correct answer

$$R(\alpha_1 | x) = 18 \cdot 0.75 + 27 \cdot 0.25 = 13.50 + 6.75 = 20.25$$

$18 \cdot 0.75 = 13.50 \text{ (ill)}$	$27 \cdot 0.25 = 6.75$
---------------------------------------	------------------------

$R(\alpha_1 | x) = 20.25$

wrong answer

The risk of one action is a sum over *all* true states. The answer 6.75 is only the healthy-patient part.

Step by step

1. Fix the action: α_1 (treat) = one column of the loss table.
2. Write one term per true state: $R(\alpha_1 | x) = \lambda(\alpha_1 | \omega_1)P(\omega_1 | x) + \lambda(\alpha_1 | \omega_2)P(\omega_2 | x)$.
3. Complete the posteriors: $P(\omega_2 | x) = 1 - 0.75 = 0.25$.
4. Insert: $18 \cdot 0.75 + 27 \cdot 0.25 = 20.25$.
5. **Sanity check:** the risk is a weighted average of the column entries, so it lies between 18 and 27.
6. If asked for the decision: $R(\alpha_2 | x) = 57 \cdot 0.75 + 0 \cdot 0.25 = 42.75 > 20.25 \Rightarrow \text{treat}$.

Remember

Risk = **weighted average** of one action's losses. It can never be smaller than the smallest loss in that column.

Valid discriminant functions

Question

Which of the following are valid discriminant functions $g_i(x)$ for the MAP rule? Candidates included $P(\omega_i)$, $\sqrt{p(x | \omega_i)P(\omega_i)}$, and $P(\omega_i | x)$.

✗ Mistake

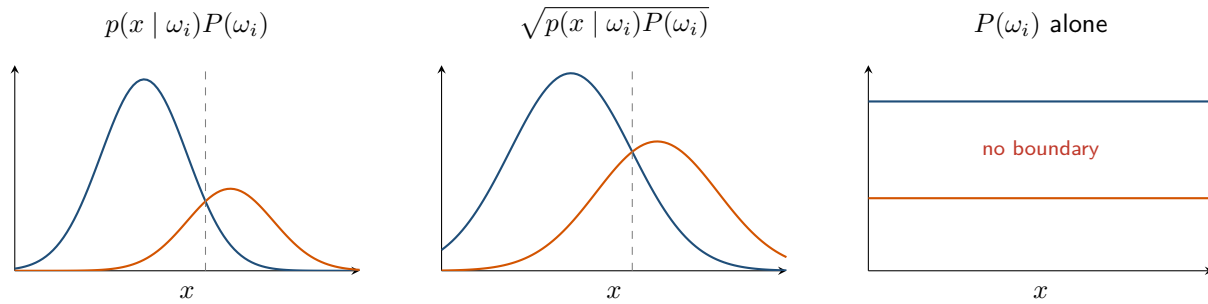
$P(\omega_i)$ and $\sqrt{p(x | \omega_i)P(\omega_i)}$: the prior alone was accepted, and the posterior itself was missed.

✓ Correct answer

$\sqrt{p(x | \omega_i)P(\omega_i)}$ and $P(\omega_i | x)$.

Why

$g_i(x)$ is a valid discriminant function if it **ranks the classes like the posterior** $P(\omega_i | x)$ for **every** x . The posterior itself qualifies trivially, and so does any strictly increasing transform applied equally to all classes. $P(\omega_i)$ ignores x : it picks the same class for every input.



Two classes with priors 0.7 and 0.3. The square root changes the curves but not where they cross, so the decision boundary (dashed) is the same. The priors alone never cross: every x goes to class 1.

Candidate $g_i(x)$	Valid?
$P(\omega_i x)$	✓ always (the reference)
$p(x \omega_i)P(\omega_i)$	✓ always (posterior $\times p(x)$, same for all classes)
$\sqrt{\cdot}, \ln(\cdot)$ of the above	✓ always (strictly increasing)
$p(x \omega_i)$	✗ only if all priors are equal
$P(\omega_i)$	✗ only if all likelihoods are identical
$1/g_i, -g_i$	✗ never (ranking reversed)

Remember

Quick test: does it use **likelihood and prior together** (or the posterior directly), combined by an increasing function? Then it is valid.

Gaussian classes: shape of the decision boundary

Question

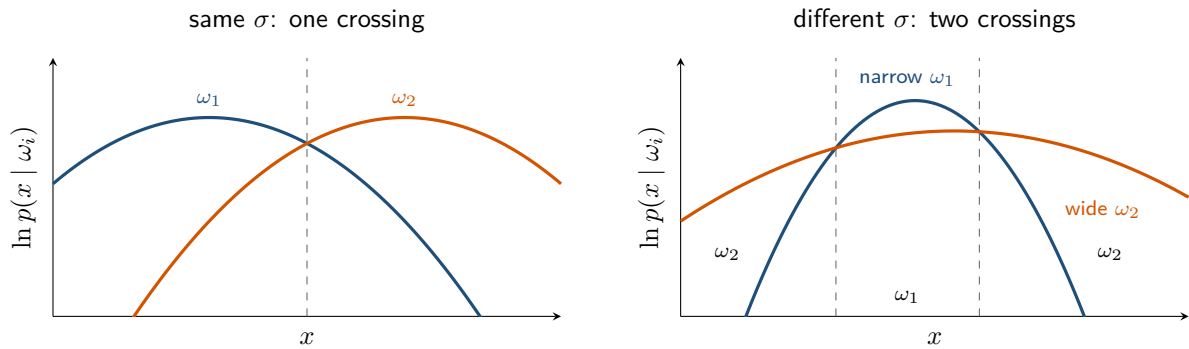
Which statements about Gaussian class-conditional densities are true? (The four statements are listed in the table.)

✗ Mistake

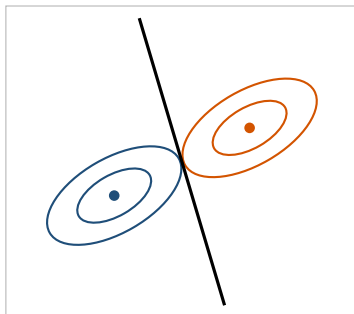
Only the last statement was ticked; the two standard facts about linear and quadric boundaries were missed.

Why

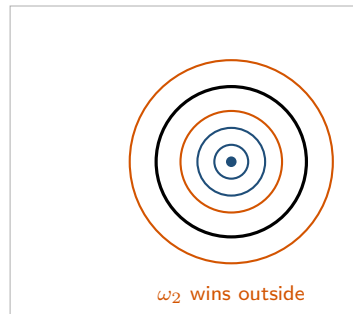
The key is one picture: **the logarithm of a Gaussian is a downward parabola**, $\ln p(x | \omega_i) = -\frac{(x-\mu_i)^2}{2\sigma_i^2} + \text{const}_i$. The width σ_i sets how sharply the parabola bends. The decision boundary is where two such parabolas (plus $\ln P(\omega_i)$) are equal, i.e. where their *difference* is zero. Two parabolas with the **same bend** differ by a straight line: the x^2 terms cancel, one crossing. Two parabolas with **different bends** differ by another parabola: the x^2 term survives, up to two crossings.



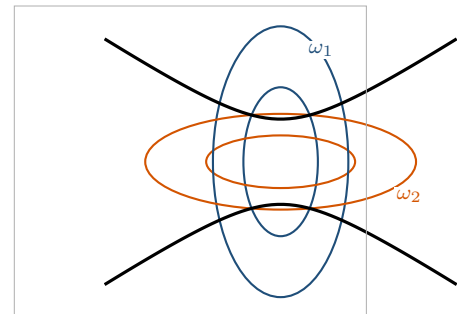
Log-densities in 1D. Left: equal widths, so the parabolas are shifted copies and cross once: a single threshold (the 1D “line”). Right: a narrow and a wide class. The narrow parabola is higher near its center but falls faster, so the wide class wins again on *both* sides: two thresholds, the 1D version of a closed quadric.



shared Σ : straight line



narrow inside wide: ellipse



crossed orientations: hyperbola

Two classes in 2D, drawn as density contours (blue ω_1 , orange ω_2), with the Bayes boundary in black for equal priors. Left: same shape and orientation, only the centers differ; the boundary is a line, tilted by Σ^{-1} (not simply perpendicular to the line between the means). Middle: same center, different spread; the narrow class wins only inside a closed curve. Right: one class elongated vertically, the other horizontally; each wins along its own long axis, and the boundary is a hyperbola.

Step by step

1. Take the log discriminant: $g_i(x) = -\frac{1}{2}(x - \mu_i)^\top \Sigma_i^{-1}(x - \mu_i) - \frac{1}{2} \ln |\Sigma_i| + \ln P(\omega_i)$.
2. Expand the quadratic form: $g_i(x) = \underbrace{-\frac{1}{2}x^\top \Sigma_i^{-1}x}_{\text{quadratic}} + \underbrace{\mu_i^\top \Sigma_i^{-1}x}_{\text{linear}} + \text{const}_i$.
3. Boundary: $g_1(x) - g_2(x) = 0$. The quadratic part of the difference is $-\frac{1}{2}x^\top (\Sigma_1^{-1} - \Sigma_2^{-1})x$.
4. $\Sigma_1 = \Sigma_2 \Rightarrow$ it vanishes $\Rightarrow$ the boundary is $w^\top x + w_0 = 0$: a **hyperplane** (line in 2D, point in 1D).
5. $\Sigma_1 \neq \Sigma_2 \Rightarrow$ it stays $\Rightarrow$ the boundary is a **hyperquadric**. Which one depends on the sign pattern of $\Sigma_1^{-1} - \Sigma_2^{-1}$: all same sign (one class narrower in every direction) $\rightarrow$ ellipse; mixed signs (each class wider in some direction) $\rightarrow$ hyperbola; zero in one direction $\rightarrow$ parabola.
6. Priors only change the constant w_0 : they *shift* the boundary toward the less probable class, never change its shape.

Statement	True?	Reason
same Σ for all classes $\Rightarrow$ linear boundary	✓	$x^T \Sigma^{-1} x$ is the same for every class and cancels
arbitrary $\Sigma_i \Rightarrow$ hyperquadric	✓	quadratic term stays; ellipse, parabola, hyperbola
same μ , different $\Sigma_i \Rightarrow g_i = P(\omega_i)$ valid	✗	likelihoods still differ with x (narrow vs. wide class)
same μ and Σ , $P(\omega_2) > P(\omega_1) \Rightarrow g_1 = 1, g_2 = 2$ valid	✓	identical likelihoods: prior decides, class 2 always wins, and $2 > 1$ ranks it first

Remember

Shared covariance $\rightarrow$ **straight** line. **Own** covariance $\rightarrow$ **curve**. Only **identical** likelihoods let the prior decide alone.

Expected classification error**Question**

Binary MAP classification. Which expressions give the expected classification error, in general and if $P(\omega_1 | x) > P(\omega_2 | x)$ for all x ?

✗ Mistake

The general min-integral was correct, but for the special case $P(\omega_2 | x)$ was chosen, which still depends on x .

✓ Correct answer

$P(\text{error}) = \int \min[P(\omega_1 | x), P(\omega_2 | x)] p(x) dx$, and in the special case $P(\text{error}) = P(\omega_2)$.

Step by step

1. If $P(\omega_1 | x) > P(\omega_2 | x)$ for every x , the classifier always says ω_1 .
2. It is wrong exactly when the truth is ω_2 . Error at x : $P(\omega_2 | x)$.
3. *Expected* error averages over all x : $\int P(\omega_2 | x) p(x) dx = P(\omega_2)$ (marginalization).
4. $P(\omega_2 | x)$ alone is the error at *one* x ; an expected value cannot still contain x .

Remember

“Expected” means the x is integrated out. If x is still in the answer, it is not an expectation.

6 Parameter Estimation

Maximum likelihood always follows the same four steps, and the result must be the *parameter*, not a statistic of the data. Judging estimators needs one formula: mean squared error equals squared bias plus variance.

Maximum likelihood: always the same recipe

The first task (Poisson) was left blank; the second (geometric) returned the sample mean instead of the parameter. Both follow the same four steps.

Question

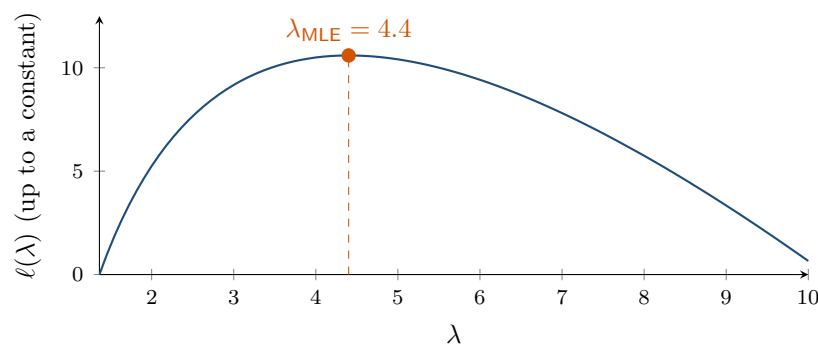
Error counts $X = \{2, 8, 4, 2, 6\}$, model $P(k) = \lambda^k e^{-\lambda}/k!$. Find λ_{MLE} .

✓ Correct answer

$$\lambda_{\text{MLE}} = \bar{k} = 22/5 = 4.40$$

Step by step

1. Likelihood (product over samples): $L(\lambda) = \prod_i \lambda^{k_i} e^{-\lambda}/k_i!$
2. Log-likelihood: $\ell(\lambda) = (\sum_i k_i) \ln \lambda - n\lambda - \sum_i \ln k_i!$
3. Derivative = 0: $\frac{\sum_i k_i}{\lambda} - n = 0 \Rightarrow \lambda_{\text{MLE}} = \frac{1}{n} \sum_i k_i$
4. Insert: $(2 + 8 + 4 + 2 + 6)/5 = 22/5 = 4.40$



The log-likelihood $22 \ln \lambda - 5\lambda$ peaks at the sample mean.

Question

Days until the first failure $D = \{8, 6, 10, 9, 4\}$, model $P(k) = (1-p)^{k-1}p$. Find p_{MLE} .

✗ Mistake

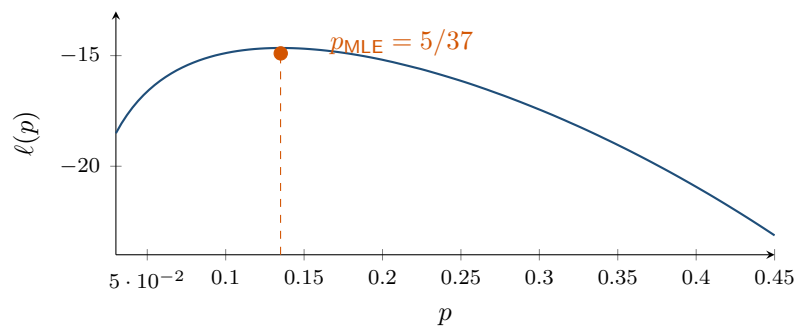
7.4: the sample mean, not the parameter.

✓ Correct answer

$$p_{\text{MLE}} = 5/37 \approx 0.14$$

Step by step

1. Likelihood: $L(p) = \prod_i (1-p)^{k_i-1} p$.
2. Log-likelihood: $\ell(p) = \sum_i (k_i - 1) \ln(1-p) + n \ln p$, with $\sum_i (k_i - 1) = 37 - 5 = 32$, $n = 5$.
3. Derivative = 0: $-\frac{32}{1-p} + \frac{5}{p} = 0 \Rightarrow 5(1-p) = 32p \Rightarrow p = \frac{5}{37}$.
4. General form: $p_{\text{MLE}} = \frac{n}{\sum_i k_i} = \frac{1}{\bar{k}}$. Insert: $1/7.4 = 0.135 \rightarrow 0.14$.
5. **Sanity check:** p is a probability. Any answer outside $[0, 1]$ is wrong.



The log-likelihood of the five waiting times peaks at one over the mean waiting time.

Model	Parameter	MLE
Poisson $\lambda^k e^{-\lambda} / k!$	rate	$\bar{k}$
Bernoulli	success probability	fraction of ones
Gaussian	μ	$\bar{x}$
Gaussian	σ^2	$\frac{1}{n} \sum_i (x_i - \hat{\mu})^2$ (divide by n)
Geometric $(1-p)^{k-1} p$	per-trial probability	$1/\bar{k}$
Exponential $\lambda e^{-\lambda t}$	rate	$1/\bar{t}$

Remember

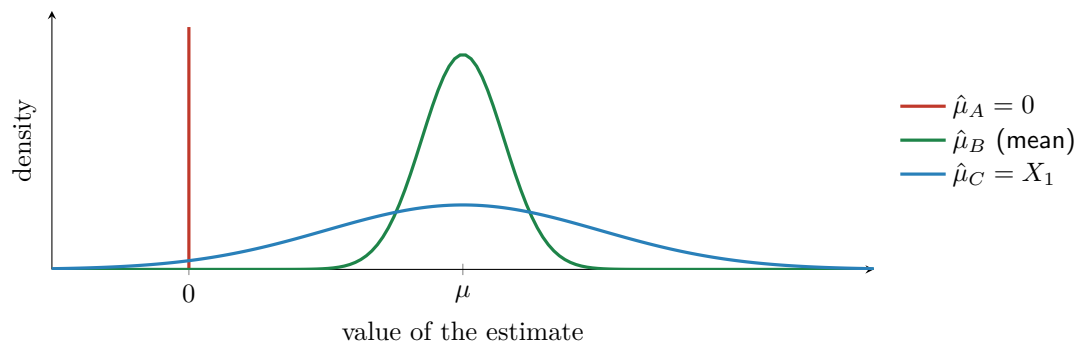
Counts $\rightarrow$ mean. **Waiting times** $\rightarrow$ one over the mean.

Bias, variance, and MSE of estimators**Question**

$X_i \sim \mathcal{N}(\mu, \sigma^2)$. Estimators $\hat{\mu}_A = 0$, $\hat{\mu}_B = \frac{1}{N} \sum_i X_i$, $\hat{\mu}_C = X_1$. Which has the highest / lowest variance, and does $\hat{\mu}_B$ always have the lowest MSE?

✗ Mistake

“ $\hat{\mu}_B$ always has the lowest MSE” was ticked. The two variance statements were correct.



Where each estimate lands over many datasets. A is a spike (no variance) at the wrong place (bias $-\mu$). B and C are centered on μ , but C scatters N times more (variance σ^2 vs. σ^2/N).

Step by step

1. Use $\text{MSE} = \text{bias}^2 + \text{variance}$.
2. Fill the table:

Estimator	Bias	Variance	MSE
$\hat{\mu}_A = 0$	$-\mu$	0 (lowest)	μ^2
$\hat{\mu}_B$	0	σ^2/N	σ^2/N
$\hat{\mu}_C = X_1$	0	σ^2 (highest)	σ^2

3. Test “always”: if $\mu^2 < \sigma^2/N$ (true mean near 0), the constant $\hat{\mu}_A$ has the lower MSE. One counterexample $\Rightarrow$ false.

Remember

A constant has **zero variance** but can have huge bias. Lowest MSE depends on the unknown μ , so “always” fails.

7 Training Neural Networks

This chapter follows one training step: what backpropagation stores and passes on, how the loss shapes the gradient, why SGD is used, and how to read and fix underfitting and overfitting. Most mistakes here were single wrong ticks among otherwise correct answers, so the tables list a verdict and a one-line reason for every statement.

Memory consumption of backpropagation

✗ Mistake

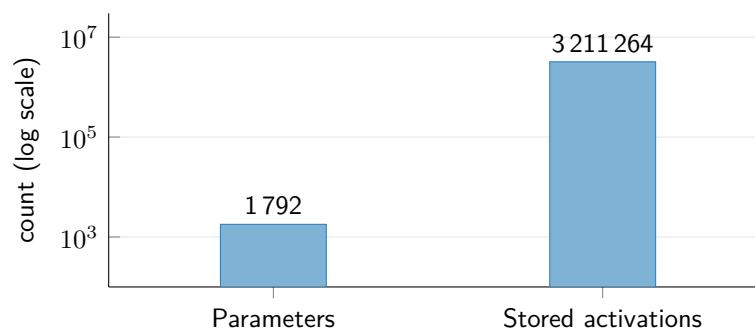
“Memory is proportional to the number of trainable parameters.”

✓ Correct answer

“Memory is proportional to the combined output dimensionalities of the nodes in the network.”

Why

The backward pass needs, at every node, the value that node produced in the forward pass: the input x for the derivative of $w \cdot x$, the ReLU input to know whether it was active. All these **activations** are stored until the backward pass. Their number is the sum of the output sizes of all nodes (times the batch size). Convolutional layers have few parameters but huge outputs, so parameter count and memory are not proportional.



One 3×3 convolution, $3 \rightarrow 64$ channels, on a 224×224 image: under two thousand parameters, but over three million output values that must be kept for the backward pass (per image).

Also: one update step needs exactly **one** forward and **one** backward pass. Statements with “multiple passes” are false.

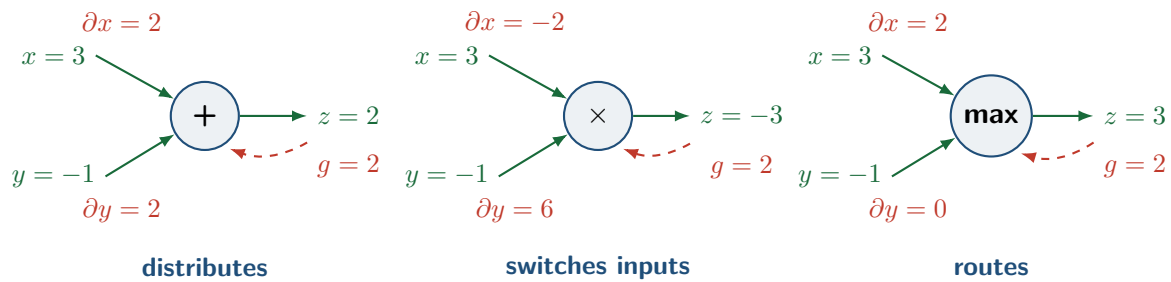
Gate rules

✗ Mistake

“Addition nodes backpropagate their inputs” was ticked as true.

✓ Correct answer

Only “Backpropagation allows for efficient optimization of computational graphs with gradient descent”.



Forward values (green) and backward gradients (red) for the same inputs $x=3$, $y=-1$ and upstream gradient $g=2$. Only the multiplication node uses the input values, and it swaps them: $\partial x = g \cdot y$, $\partial y = g \cdot x$.

Node	Forward	Backward (upstream gradient g)
Addition $z = x + y$	sum	$\partial x = g$, $\partial y = g$ (copy the gradient)
Multiplication $z = xy$	product	$\partial x = g y$, $\partial y = g x$ (swap the inputs)
Max $z = \max(x, y)$	maximum	g to the larger input, 0 to the other

Why

The addition node passes on the *gradient*, never its inputs. Backpropagation also does *not* reduce SGD noise (that comes from mini-batches) and does *not* guarantee the optimum (the loss is non-convex). It only computes the exact gradient efficiently.

Gradients of squared and absolute error

Question

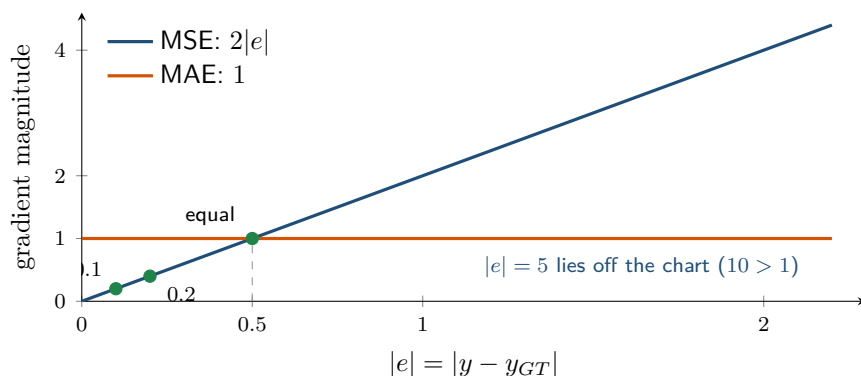
$l_1 = (y_{GT} - y)^2$ (MSE), $l_2 = |y_{GT} - y|$ (MAE). For four pairs (y, y_{GT}) , compare $|\partial l_1 / \partial y|$ with $|\partial l_2 / \partial y|$.

✗ Mistake

Two cases were judged by the size of the *error* instead of the size of the *gradient*: for $|e| = 0.2$, “MSE gradient larger” was accepted, and for $|e| = 0.1$, “MSE gradient smaller” was rejected.

✓ Correct answer

MSE gradient larger for $|e| = 5$; equal for $|e| = 0.5$; smaller for $|e| = 0.2$ and $|e| = 0.1$.



The MSE gradient grows with the error, the MAE gradient stays at 1. They cross at $|e| = 0.5$. Below that point MAE pushes harder, above it MSE.

Step by step

1. Differentiate: $\frac{\partial}{\partial y}(y_{GT} - y)^2 = -2(y_{GT} - y)$, magnitude $2|e|$. $\frac{\partial}{\partial y}|y_{GT} - y| = \pm 1$, magnitude 1.
2. Compare $2|e|$ with 1 for each case:

y	y_{GT}	$ e $	MSE $2 e $	MAE	claimed	
5	10	5	10	1	>	✓
1.2	1	0.2	0.4	1	>	✗
6.5	7	0.5	1	1	=	✓
0.1	0	0.1	0.2	1	<	✓

Remember

Threshold $|e| = \frac{1}{2}$ (for e^2 without a factor $\frac{1}{2}$). With $\frac{1}{2}e^2$ the gradient is $|e|$ and the threshold moves to 1.

Reasons for SGD**✗ Mistake**

“SGD converges with fewer steps” was ticked; “a batch gradient points approximately in the direction of the full gradient” was missed.

Statement	True?	Reason
fewer steps than gradient descent	✗	noisier steps, usually <i>more</i> of them; each step is cheaper
batch gradient $\approx$ direction of full gradient	✓	unbiased estimate of the full gradient
cost per step independent of dataset size	✓	depends on batch size only
lower training error	✗	no such guarantee

Universal approximation**✗ Mistake**

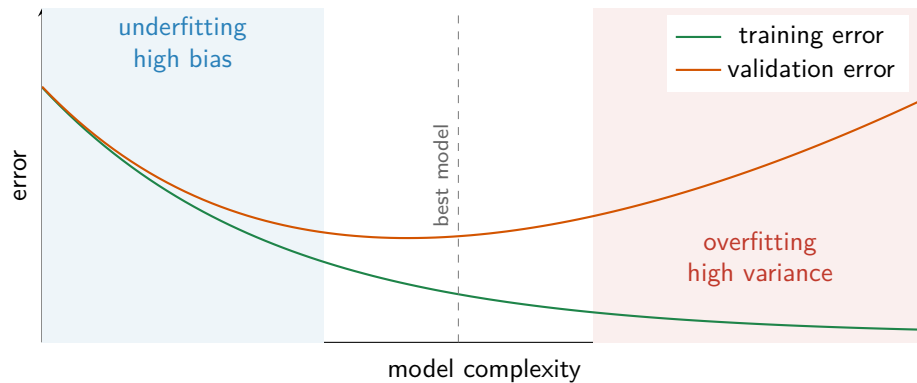
“A neural network can approximate *any* function” was ticked as true.

Statement	True?	Reason
can approximate <i>any</i> function	✗	only continuous functions on a compact domain; not discontinuous ones
theorem says how to find optimal parameters	✗	pure existence statement
can approximate non-differentiable functions	✓	$ x $ is continuous; ReLU nets are piecewise linear
can approximate any continuous function on a compact set	✓	the theorem itself

Bias, variance, and model complexity

✗ Mistake

“A model with high bias tends to underfit the data” was missed.



Training error always falls with complexity. Validation error falls, then rises again. High bias sits on the left (both errors high), high variance on the right (small training, large validation error).

Statement	True?	Reason
Small training error $\Rightarrow$ sufficient complexity	✓	the model is flexible enough to fit the data
Small validation error $\Rightarrow$ good performance	✓	unseen data estimates generalization
Small training error $\Rightarrow$ good performance	✗	the model could overfit (right side of the plot)
High bias $\Rightarrow$ underfitting	✓	too simple, it misses even the training data (left side)

Remember

Bias = **B**elow the data (underfit). Variance = **V**olatile, fits the noise (overfit).

Fixing underfitting

✗ Mistake

“More data” was ticked; “wider architecture” was missed.

underfitting
train error high
wider, deeper,
train longer

overfitting
train low, val. high
more data, regularize,
early stopping

Different problems, different cures. More data fights overfitting; an underfitting model already fails on the data it has. Batch normalization helps optimization but adds no capacity.

Remember

Underfitting = too **little capacity**: add units (wider) or layers (deeper). “More data” is the cure for the *other* problem.

8 Convolutional Networks

A convolutional layer is defined by its kernel tensor: kernel size, input channels, output channels. Parameter counts, width versus depth, and the comparison with MLPs all follow from this tensor; pooling adds invariance on top.

Number of weights

Question

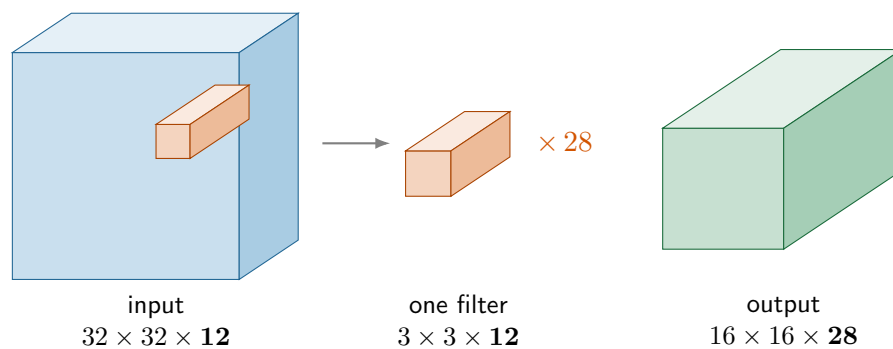
A 3×3 convolution maps a $32 \times 32 \times 12$ input to a $16 \times 16 \times 28$ output. How many weights (without biases)?

✗ Mistake

$252 = 3 \cdot 3 \cdot 28$: the input channels were forgotten.

✓ Correct answer

$$3 \cdot 3 \cdot 12 \cdot 28 = 3024$$



Every filter spans the *full input depth* ($3 \times 3 \times 12 = 108$ weights). One filter produces one output channel, so 28 filters are needed. The spatial sizes ($32 \rightarrow 16$) do not enter the count; they only reveal the stride.

Step by step

$$\# \text{weights} = K_h \cdot K_w \cdot C_{\text{in}} \cdot C_{\text{out}} = 3 \cdot 3 \cdot 12 \cdot 28 = 3024, \quad \# \text{biases} = C_{\text{out}} = 28.$$

Stride: $W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - K}{S} \right\rfloor + 1$. Halving the size ($148 \rightarrow 74$ with $K = 3$, $P = 1$) means $S = 2$.

More filters in one layer

✗ Mistake

“Higher abstraction capabilities of the layer” was ticked as an effect of adding filters.

✓ Correct answer

Increase in output dimensionality; increased number of features the layer can respond to.

Change	What it adds	Abstraction?
More filters (width)	more output channels = more feature detectors at the same level	✗
More layers (depth)	larger receptive field, features of features	✓

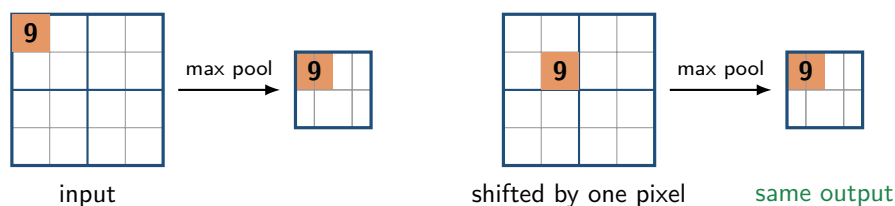
Effects of max pooling

✗ Mistake

“Translation equivariance” was ticked instead of “translation invariance”.

✓ Correct answer

Dimensionality reduction, translation invariance, loss of spatial/relational information.



A small shift inside a 2×2 pooling block does not change the pooled output: **invariance**. The price is that the exact position is lost. An *equivariant* operation (convolution) would shift its output along with the input.

Remember

Conv = Equivariant (output moves with the input). **Pool = Invariant** (output stays).

True for CNNs but not for MLPs

✗ Mistake

“Batch normalization improves stability and convergence” was ticked, although it holds for MLPs too; “more parameters do not necessarily mean more features” was missed.

Step by step

For every statement ask two questions: true for CNN? *and* false for MLP? Only “yes, yes” counts.

Statement	CNN	MLP	Reason
translation equivariant	✓	✗	shared kernel slides; MLP weights are position specific
batch norm stabilizes training	✓	✓	works for any layer type
no activations $\Rightarrow$ n layers = 1 layer	✓	✓	composition of linear maps is linear
more parameters $\nrightarrow$ more features	✓	✗	larger kernel adds weights, not channels; in an MLP more weights need more units

9 Recurrent Networks

Backpropagation through time is ordinary backpropagation on the unrolled network. The only question is how many paths lead from the output to the variable of interest: an input enters at one time step, the shared weight at every time step.

Derivative with respect to an input

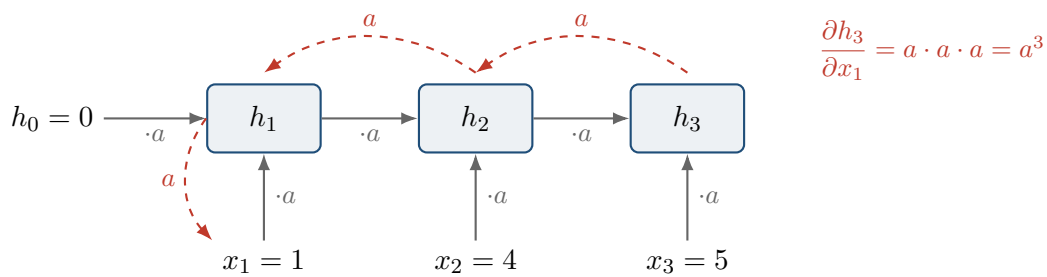
This task was left blank.

Question

RNN cell $h_t = a h_{t-1} + a x_t$ with $a = 1$, $h_0 = 0$, sequence $(x_1, x_2, x_3) = (1, 4, 5)$. What is the derivative with respect to x_1 ?

✓ Correct answer

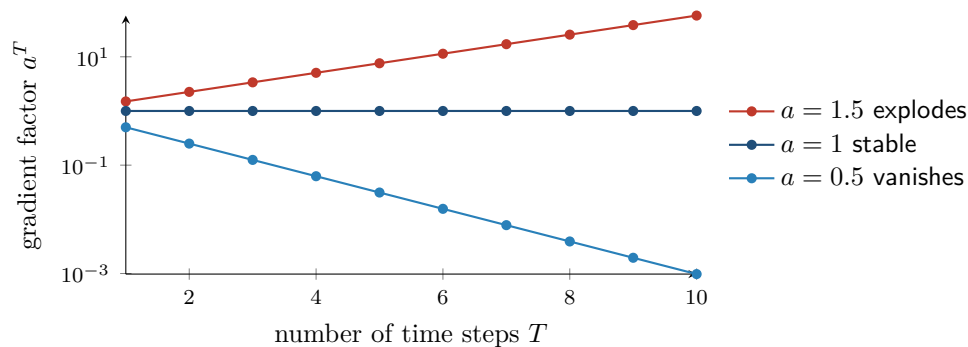
$$\partial h_3 / \partial x_1 = a^3 = 1$$



Unrolled over three time steps with shared weight a . The gradient travels backwards (red) and picks up one factor a per step.

Step by step

1. **Unroll:** $h_1 = ax_1$, $h_2 = a^2x_1 + ax_2$, $h_3 = a^3x_1 + a^2x_2 + ax_3$.
2. **Chain rule backwards through time:** $\frac{\partial h_3}{\partial x_1} = \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial x_1} = a^3$.
3. **Insert $a = 1$:** result 1. The values $x_2 = 4$, $x_3 = 5$ are distractors, because the cell is linear.



The same product a^T explains the vanishing and exploding gradient problem of RNNs.

Derivative with respect to the shared weight

Question

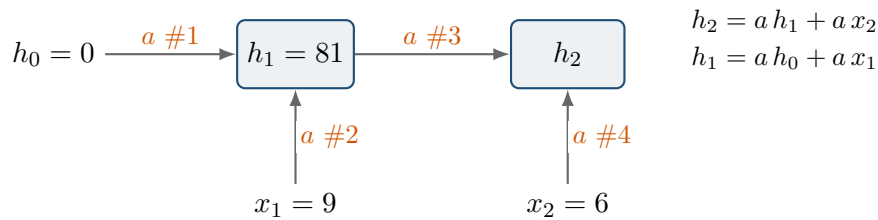
Cell $h_t = a h_{t-1} + a x_t$, $a = 9$, $x_1 = 9$, $x_2 = 6$, $h_0 = 0$. Find $\partial h_2 / \partial a$.

✗ Mistake

81: this is h_1 , only one of the paths through which a acts.

✓ Correct answer

$$\partial h_2 / \partial a = 2a x_1 + x_2 = 162 + 6 = 168$$



The weight a is used four times. The derivative collects one contribution from every use; the answer 81 stops after one of them.

Step by step

- Fast way, unroll to a closed form:** $h_1 = a x_1$, $h_2 = a h_1 + a x_2 = a^2 x_1 + a x_2$.
- Differentiate as an ordinary polynomial in a : $\frac{\partial h_2}{\partial a} = 2a x_1 + x_2 = 2 \cdot 9 \cdot 9 + 6 = 168$.
- Check with the recursive BPTT rule** (product rule, since h_{t-1} also depends on a):

$$\frac{\partial h_t}{\partial a} = h_{t-1} + x_t + a \frac{\partial h_{t-1}}{\partial a}.$$

$$t = 1: 0 + 9 + 9 \cdot 0 = 9. \quad t = 2: 81 + 6 + 9 \cdot 9 = 168.$$

Asked for	Formula (this cell)	Pitfall
$\partial h_T / \partial x_1$ (input)	a^T	one path only
$\partial h_2 / \partial a$ (shared weight)	$2a x_1 + x_2$	sum over <i>all</i> uses of a

Remember

Derivative w.r.t. an **input**: one chain of factors. Derivative w.r.t. the **shared weight**: a **sum** over every time step (product rule). Safest: unroll to a formula in a , then differentiate.

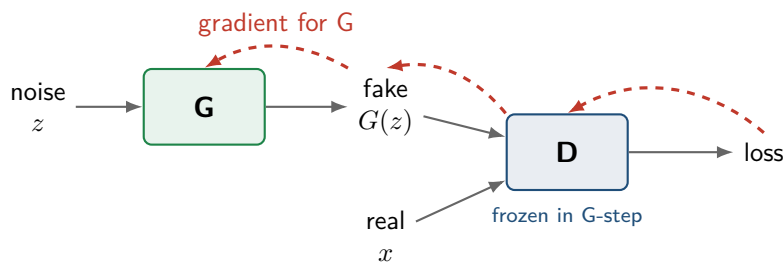
10 Generative Adversarial Networks

A GAN couples two networks through one loss. The mistake here was to treat the generator as if it were trained separately from the discriminator; the table also covers the related trap of reading the generator as a density estimator.

How the generator is trained

✗ Mistake

“The computation of backpropagation for the generator is independent of the discriminator” was marked as true.



The generator's loss is measured at the discriminator's output. Its gradient must travel back through D (with D's weights frozen) to reach G. Without D there is no training signal for G.

Statement	Answer	Reason
Generator learns the (empirical) probability function	✗ False	G is an <i>implicit</i> model: a sampler $z \mapsto x$, not a density you can evaluate
All GANs require labeled data (x, y)	✗ False	the basic GAN needs only real images; real/fake labels are generated automatically
CycleGAN trains two generators and two discriminators	✓ True	$G: X \rightarrow Y$, $F: Y \rightarrow X$, plus D_X , D_Y and the cycle loss
Generator backprop is independent of the discriminator	✗ False	the gradient flows through D (figure above)

Remember

Explicit density estimators: MLE, Parzen windows, (restricted) Boltzmann machines. A GAN only *samples*. Check whether the question asks for “estimating a distribution” or “generating samples”.

11 Checklist Before Submitting

One line per habit, sorted by chapter. Run through the list in the last minutes of the exam.

Multiple choice

- ☐ Every option judged on its own, including the “easy” ones?
- ☐ “always”, “only”, “any”, “fewer” tested with one counterexample?
- ☐ “A but not B”: both halves checked for every statement?
- ☐ Partner concepts not swapped (invariance/equivariance, width/depth, gradient/input)?
- ☐ Numeric: formula with all terms written first; result plausible?

Vision tasks and pipeline

- ☐ System tasks: every output mapped to its task (identity = recognition)?
- ☐ Pipeline stage chosen by input/output type (image in, image out = data prep.)?
- ☐ Detector: sliding window, pyramid, boxes *and* NMS?

Histograms, entropy, saliency

- ☐ Histogram axis orientation fixed with the unambiguous patch?
- ☐ Entropy only depends on bar heights (swap, shuffle, mirror keep it)?
- ☐ Saliency: entropy *peak over scale*, not maximum entropy?

Hough and template matching

- ☐ $\rho = x \cos \theta + y \sin \theta$ computed for every candidate?
- ☐ Votes = product of bins of the *free* parameters only?
- ☐ R-table: orientation range (mod 180° or mod 360°) checked?
- ☐ Correlation theorem: exactly one conjugate?

Bayesian decisions

- ☐ Evidence = sum over *all* hypotheses; round only at the end?
- ☐ Risk: one term per true state; result between the column’s min and max loss?
- ☐ Discriminant: likelihood *and* prior (or the posterior), increasing transform?
- ☐ Shared $\Sigma \rightarrow$ linear, own $\Sigma_i \rightarrow$ quadric?
- ☐ “Expected” value: is x integrated out?

Estimation

- ☐ MLE: log-likelihood, derivative, solve; answer is the parameter?
- ☐ Probability inside $[0, 1]$? Waiting times $\rightarrow 1/\bar{k}$?
- ☐ $\text{MSE} = \text{bias}^2 + \text{variance}$; “always lowest MSE” tested?

Neural networks

- ☐ Backprop memory $\propto$ stored activations, not parameters?
- ☐ MSE vs. MAE gradient: compared $2|e|$ with 1?
- ☐ SGD: cheaper steps, not fewer steps?
- ☐ Universal approximation: continuous on compact domain, existence only?
- ☐ Underfitting $\rightarrow$ wider/deeper; overfitting $\rightarrow$ more data/regularize?

CNNs

- ☐ Conv parameters: $K \cdot K \cdot C_{\text{in}} \cdot C_{\text{out}}$, spatial size irrelevant?
- ☐ Pooling = invariance, convolution = equivariance?

RNNs and GANs

- ☐ BPTT w.r.t. input: one factor per step; w.r.t. weight: sum over every use?
- ☐ GAN: generator gradient flows through the discriminator?